

dress

1. Market segment requirements (e.g. DevOps platform solution). Ours are outlined below with more details here

1. Industries/verticals requirements (e.g. Pub Sec, High Tech, Retail)

1. Business segment requirements (e.g. enterprise vs SMB)

Relevant solutions will differ by audience.

- C-level executives care about broad business solutions, often characterized by well-funded strategic business initiatives - things that help them improve productivity, reduce cost, or deliver value faster - things that align with Command of the Message Customer Value Drivers. The C-level audience are important influencers who set an organizations agendas, strategic objectives, and fund major IT initiatives.

- DevOps buyers are tasked with determining HOW to meet the transformational business objectives backed by their C suite. They likely have a budget to buy or replace specific DevOps capabilities (like CI) and the more Directors and VPs often look at the bigger picture of how to simplify a complex DevOps environment.

- DevOps users are tasked with using the tools to make the transformation happen. They care about how a given tool makes their job easier, their time more productive, and their code more secure.

A picture may help convey the Solution Framework.

One of the most common of these sweeping initiatives is Digital Transformation. The Enterprise Project provides a useful definition of Digital Transformation: "Digital transformation is the integration of digital technology into all areas of a business, fundamentally changing how you operate and deliver value to customers."

A Digital Transformation initiative is typically executed through a set of strategic business solutions. Research in November 2021, showed that C-level executives are concerned most with these Digital Transformation Solutions:

| Strategic Solution | primary objective | exec sponsor | characteristics |

| ---- | ---- | ----- | --- |

| 1. Security and compliance | understand and manage risk | CISO or CTO | requires technology and process transformations |

| 1. Cloud transformation | gain business agility | line-of-business VP (LOB) or CTO | often technology-focused with adoption of cloud, containers, Kubernetes, IaC |

| 1. Application modernization | improve software time-to-market | CTO | often process-focused with DevOps, Agile, velocity, VSM |

Connecting solutions to value plays

These transformative solutions must get connected to DevOps initiatives where GitLab is established. It's important to guide prospects from broad ideas down to specific actions, budgets, and sales opportunities. We do this via core DevOps GTM solutions where we articulate a message of how we help customers achieve positive business outcomes, and where we align sales

efforts (Value plays and revenue programs) and marketing campaigns (such as digital ads, email campaigns, nurture, events), to focus on opportunities where we have the greatest likelihood of success. By providing a solutions framework and capturing steps in value plays, we expect to achieve repeatable, predictable sales outcomes with metrics that help us optimize marketing spend for greatest impact.

GitLab customers can purchase these solutions by working directly with our sales teams or by working with our Channel Partners and Technology Alliance Partners. Partners support GitLab customers by offering services that deliver DevOps solutions. The core DevOps solutions and the value plays each supports include:

- Software Security
- Software Compliance
- Automated Software Delivery
- and cumulatively, The DevSecOps Platform.

Which solution to use for which persona

The core DevOps solutions are communicated to sales via Solutions Selling in HighSpot.

Understanding which solution to apply where requires consideration of the persona/audience. The matrix below suggests which conversation is best for which persona as a recommended starting point.

Persona / Primary initiative	Security & Compliance	Cloud Transformation	App Modernization
----	----	----	----
CIO, CTO, CISO	Software Compliance	Platform	Platform
Dev/LOB	DevSecOps	Automated Software Delivery	Automated Software Delivery, DevSecOps
Security	DevSecOps	Software compliance	DevSecOps, Software compliance
Ops	Software compliance	Platform	Automated Software Delivery
Risk/legal/CFO	Software compliance	Software compliance	Software compliance

DevOps Solution DRIs and key links

	Automated Software Delivery	Security/DevSecOps	Compliance	Platform
---	-----	-----	-----	-----
PMM	TBH Saumya Upadhyaya as placeholder	Brian Mason	Brian Mason	Saumya Upadhyaya
TMM	Itzik	Fern	Fern	William Arias
PM	Jackie Porter	Hillary Benson	Hillary Benson	Mike Flouton / Justin Farris
Resource page	Automated Software Delivery	DevSecOps page	Compliance	Platform
Solution page	Automated Software Delivery solution	DevSecOps solution	Compliance solution	GitLab home page
Highspot page	Automated Software Delivery	Software Security	Software Compliance	DevSecOps Platform

Use cases

Within the core DevOps solutions, are use cases. These topics may be hot in the market (like Security), or associated use cases with GitLab (like SCM). They may attract interest for a

conversation with a prospect. Conversations about these capabilities may open the door (as a wedge) to broader conversations that align to value plays - typically leading to either the platform solution or Automated Software Delivery solution.

Examples include:

- Source Code Management (SCM).
- Continuous integration
- GitOps

The Customer Success Managers (CSMs) are goaled on expanding an existing customer's adoption of GitLab use cases, as measured by use case adoption. Mapping use cases to stages helps the CSMs know which use case material to apply.

Use cases DRIs and key links

Use case	PMM	PM	Handbook	Adoption guide	Highspot page
-----	-----	-----	-----	-----	-----
SCM	Aathira Nair	Derek Ferguson	link	link	Part of Automated Software Delivery play
CI	Daniel Hom / Saumya Upadhyaya (placeholder)	Jackie Porter	link	link	Part of Automated Software Delivery play
CD	Daniel Hom	Viktor Nagy	link	link	Part of Automated Software Delivery play
Security	Brian Mason	Sarah Waldner	link	link	Security and Governance play
Compliance	Brian Mason	Sam White	link	link	Security and Governance play
Artifact Mgmt	Daniel Home / Saumya Upadhyaya (Placeholder)	Tim Rizzi	link	link	Part of Automated Software Delivery play
Agile Planning	Aathira Nair	Melissa Ushakov	link	link	link
Analytics & Insights	Aathira Nair	Justin Farris	link	link	link
Platform Engineering	Saumya Upadhyaya	Justin Farris	link	link	link

Future Use cases

- Test automation (UI test automation, mobile app testing, performance tests, load tests ..)
- GitOps
- Ecosystem - extensibility, plugins, content
- Observability
- AI (currently AI workflows will be captured as part of the usecases they are part of. Future decision to be made to track as a separate usecase)

Last mile alignment

For optimal impact, sales and marketing efforts must align across the entire sales and marketing life cycle.

Internally, solutions GTM should drive:

- Value plays which align with revenue programs
- campaigns which support value plays
- Highspot and Path Factory topics/spaces
- metrics fields in SFDC Command Plan for proper reporting and assessment

Externally, solutions GTM should drive:

- Solutions pages (from Home page drop-down)

Buyer's journey and typical collateral

As a prospect is determining that they have a specific problem to solve, they typically go through several stages of a buyer's journey which we hope ultimately leads them to selecting GitLab as their solution of choice.

| Awareness | Consideration | Decision/Purchase |

| ----- | ----- | ----- |

| Collateral and content designed to reach prospects in this stage of their journey should be focused on educating them about the problems they are facing, the business impact of their problems, and the reality that others are successfully solving the same problem | Collateral and content designed to reach prospects in this stage of their journey should be focused on positioning GitLab as a viable and compelling solution to their specific problem. Use cases can address unique concerns and may help overcome perceived barriers. | Collateral and content designed to reach prospects in this stage of their journey should be focused on key information that a buyer needs to justify GitLab as their chosen solution. |

| Typical Awareness Collateral | Typical Consideration Collateral | Typical Decision/Purchase Collateral |

| - White papers describing the problem space
- Infographics illustrating the impact of the problem/challenge
- Analyst reports describing the problem/domain
- Webinars focusing on the problem and how can be solved
- Troubleshooting guides to help overcome the problem
- Analysis of public cases where the problem impacted an organization (i.e. outage, data loss, etc) | - White papers describing the innovative solutions to the problem
- Infographics illustrating the success and impact of solving the problem
- Analyst reports comparing different solutions in the market (MQ, Waves, etc)
- Webinars focusing on the success stories and how GitLab helped solve the problem
- Customer Case Studies, Videos, Logos, etc
- Solution Check Lists / Plans for how to solve the problem
- Comparisons between GitLab and other solutions | - ROI calculators
- Use case specific implementation guides
- Use Case migration guides (from xyz to GitLab)
- Getting Started info
- References and case studies |

Buyer's Journey - Audience

In addition the buyer's journey, there are also different audiences in an organization with different needs at different times. The general model for the buyer's journey outlines kinds of collateral that is needed at different stages of the buying cycle. It is incredibly important

to understand the needs of the audience when creating collateral. Executives, managers, and individual contributors will need different information to support their specific work.

- Executives will need information about business value, risk, cost, and impact to support strategic objectives
- Managers will need more detailed information to drive planning, justification, and migration details.
- Contributors need technical details about how it works, how it's better, and how it's going to help them in the future.

The Buyer's journey can also start when developers and the team start to look for solutions to their day to day challenges. The staff initiates the effort, as they have identified a challenge and are the champion of a change. They then convince their managers and executives of the need to consider an alternative to their current state.

Solution GTM Bill of Materials

To this model, we need to add ideal customer profiles to better inform marketing on types of companies to target for campaigns and to help sales qualify opportunities.

Market Requirements

Solution "Market Requirements" are the broad capabilities that a solution (product or multiple products) would need to support. Each GitLab DevOps solution should have roughly 8-12 market requirements defined.

In product terminology, Market Requirements are similar to concept of Jobs to be Done (JTBD). Both represent something the end user or customer wants to accomplish. Like JTBD, Market Requirements are *not* features but instead a set of higher level capabilities that indicate what's "required" for a given solution to include in order to be successful in the market. For example, specific products do contain features that help to meet Market Requirements but the features themselves don't qualify as Market Requirements. JTBD tend to be very granular and specific. The list of JTBD for a particular stage can be long and is theoretically endless. Market Requirements are intentionally broad in an effort to keep the list of total requirements small and consumable. We expect that a given Market Requirement would consist of many jobs to be done

In Command of the Message terminology (internal-only) used by Sales, these are NOT the same thing as "Required Capabilities". COTM "Required Capabilities" are very narrow and focused on a specific value driver aligned with GitLab. "Market Requirements" are intended to be comprehensive for a given use case and defined by the market (analysts, vendors, etc), not GitLab specific features), as well as seen through an 'outside-in' paradigm of the market.

When industry analysts do research, they use a market requirements model to evaluate competing solutions. A typical analyst report may have a set of 40-60 capabilities in their market requirements model used to measure and compare vendors with each other. For the sake of our go-

to-market efforts, we bucket capabilities into 8-12 key market requirements per use case.

The downstream impacts of market requirements are

- Technical Demos are based on showcasing specific market requirements
- Comparisons across the market (multiple vendors, multiple products) can be driven from market requirements
- ROI models can be built highlighting the value of specific market requirements.

<!--

Solution GTM Bill of Material Priorities & Tracking (needs updating so commented out for now)

<div class="wide-sheet">

<iframe width="1200" height="1100" src="https://docs.google.com/spreadsheets/d/e/2PACX-1vTptvo1N7m8tjLSgJFOoy8l2fhnJmlyS83JhsT2DU6Ki3x1tcJ0Jd4Y4nvDnz0EbLEddAzpWHQYMWVs/pubh80560927&gridlines=false&single=true&widget=true&headers=false&rm=minimal∓range=A1:L42"></iframe>

</div>

- GTM Tracking Spreadsheet (requires updating)

-->

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/usecase-gtm/bom.md

What's on this page?

This page lists the details of the Technical Marketing Bill of Material deliverables

Use case demo video

Description: Problem to solve and how we solve it (<=10 mins)

Quantity: 1

Buyer's Journey: Awareness

Priority: 1

Use case demo click-through/sim demo

Description: Interactive version of demo video

Quantity: 1

Buyer's Journey: Awareness

Priority: 1

How to get started videos

Description: For GitLab Learn. (<= 5 mins each)

Quantity: 3
Buyer's Journey: Awareness
Priority: 1

Technical Blog plan

Description: 12 month plan - 2/quarter - identify subjects
Quantity: 1
Buyer's Journey: Awareness
Priority: 1

Technical White paper

Description: How to do xyz use case
Quantity: 1
Buyer's Journey: Consideration
Priority: 2

Technical Deck

Description: For webinars/conferences
Quantity: 1
Buyer's Journey: Consideration
Priority: 2

Differentiator demos

Description: For competitive page
Quantity: 2-3
Buyer's Journey: Decision
Priority: 2

POV Template tasks

Description: Partner with SA team to create key tasks/deliverables for a Proof of Value (see POV example: <https://docs.google.com/document/d/1ZJ4Yt0-ja-LH0DZLLn1tbWIKcSafgN4UQJ4Z1CtPuo/edit#heading=h.rei57tajtrnc>)
Quantity: 1
Buyer's Journey: Decision
Priority: 3

Technology partner demo

Description: Show how GitLab and partner technology work together (where appropriate)
Quantity: 1
Buyer's Journey: Consideration
Priority: 3

Trial quick start guide

Description: Expand initial 3 getting started videos into a full blown trial quick start guide
Quantity: 1
Buyer's Journey: Consideration
Priority: 3

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/usecase-gtm/compliance.md

Looking for a customer-facing overview of GitLab's Software Compliance capabilities? See the Compliance Solution

GitLab field teams refer to this Security and Governance highspot page for latest information

The page below is intended to align GitLab sales and marketing efforts with a single source of truth for our go-to-market efforts around DevSecOps.

Who to contact

	Product Marketing		Developer Advocate	
	-----		-----	
	Brian Mason (@BrianMason)		Fernando Diaz (@fjdiaz)	

The Market Viewpoint

Software compliance

The Software Compliance solution is applicable for customers who are concerned about securing their software supply chain and simplifying their compliance with common industry regulations while at the same time speeding their software velocity.

GitLab's platform approach seamlessly embeds security and compliance within the DevOps platform, providing simplicity, visibility, and control.

Why is compliance a new concern?

While compliance and auditability has always been important, these requirements have greater attention following high-profile attacks on software supply chains and the related US President's Executive Order to improve cyber security. The Executive Order (EO) directs NIST to publish preliminary guidelines for enhancing software supply chain security by November 8, 2021. It is anticipated that NIST, the National Institute of Standards and Technology, will provide even more guidance. This article explains clearly why it's important to anyone developing software. It says, while the EO is "directly applicable to the U.S. Government, the standards being established for U.S. Government agencies could be adopted as industry standards for all organizations that develop or acquire software similar to various industries adopting the NIST Cybersecurity Framework as a security controls baseline."

Application security testing is still a foundational part of compliance, but now visibility and control across the entire software factory is even more paramount than before, a capability

that is made challenging by complex DevOps tool chains.

Desired business outcomes

- Meet industry regulatory requirements - efficiently and without slowing velocity of development
- Secure the software supply chain with better end-to-end governance to reduce security and compliance risks

Personas

User Personas

Cameron the Compliance Manager needs to be sure all the company's development processes are compliant. Given the amount of data that a software development and delivery lifecycle produces, and the complexity of typical DevOps tool chains, he finds it difficult to find, aggregate, and report on all of the necessary data and changes made across systems for audit purposes. He needs to easily see who changed what, where, and when from end-to-end across the SDLC. He needs the information to be available quickly and easily so he can reduce the time and disruption involved in the evidence collection process.

Sasha the Software Developer uses GitLab primarily within the MR pipeline report. The developer cares about compliance and security but does not want to become a compliance expert. Capabilities that help them run fast while staying compliant are appreciated.

Amy the Application Security Engineer may be tasked with automating and reporting on compliance policies so would like them to be simple, efficient, and automated wherever possible. Amy finds it difficult to control which policies are applied within the development process (e.g. CI pipelines) and to keep them from being circumvented. In fact, she may not even know they've been circumvented.

Buyer Personas

The CTO or head of DevOps Architecture is usually the buyer for when compliance is the primary driver.

The key capability that addresses the CTO's need is Compliant Pipelines and the Compliant workflow automation. They need the ability to prescribe scans and policies in the CI pipeline and ensure individual developers cannot bypass them.

The Security Manager or CISO (Sam's boss) is usually the buyer for the Ultimate tier when security gets involved.

The key to winning their hearts is to focus on Simplicity and control

- Complexity is one of the CISO's chief complaints. Using one tool to provide visibility and control across the entire SDLC is valuable.
- CISOs likely feel out of control, or at least pressured to be secure amidst evolving threats, high-profile cyber attacks, new compliance concerns, and development tool sprawl. It's hard to manage these software risks. GitLab's single platform that provides end-to-end simplicity,

visibility, and control they need.

Industry Analyst Coverage

Analysts have not identified a market segment for software compliance. They have been writing articles about it though. Forrester spoke about the Executive Order at GitLab's Commit event in Fall of 2021.

Market Requirements

Market Requirements	Description	Typical capability-enabling features	Value/ROI
Common compliance controls	Controls necessary to protect the integrity of the software development and deployment process	Role-based access, MR approvals, and many others	Simplify audit and compliance and reduce risk of noncompliance.
Automated policy enforcement	Automation can reduce the audit burden. Enforcing policies within the MR shifts compliance left where developers can resolve problems early in the life cycle	locked CI templates that enforce policies in the pipeline	Avoids late rework. In regulated industries, there is an approved change order window. if it is missed for rework, the change management process must start over.
Audit reporting	Audit events should be automatically captured and reported. Changes to code, controls, and IaC should be traceable and captured as audit events across the entire SDLC.	Audit events, audit reporting	reduce risk of non-compliance and efficiently identify root causes following a security or compliance incident
Security Governance	The solution must automatically apply security policies against code changes to ensure that only appropriate risks are taken. Application vulnerabilities, representing risk, are tracked, managed, and reported. The solution must enable routine assessments of security practices to evaluate for risk, compliance, audit and process improvement opportunities (usually for education purposes).	Security policy automation, Risk and compliance reporting, Audit reporting, Variety of security metrics and process reporting, Vulnerability database and management	Efficiently monitor, manage and mitigate risk. Ability to identify exceptions and refine policies over time.
Security guardrails (Preventative - Pre CI/CD)	Preventative Application Security uses guardrails to help teams consistently build things that are secure from the start.	Compliant pipelines that cannot be circumvented by a developer, pre-approved code libraries, and auto-discovery that catalogs all third party code.	Prevents creating new vulnerabilities.

The GitLab Solution

How GitLab Meets the Market Requirements

GitLab Software Compliance solution overview

<figure class="videocontainer">
 <iframe width="560" height="315" src="https://www.youtube.com/embed/QV2dlocn-hk" frameborder="0" allow="autoplay; encrypted-media" allowfullscreen></iframe>
</figure>

Market Requirements	How GitLab Delivers	GitLab Category	Demos

Common compliance controls	GitLab provides many common controls throughout the SDLC, Audit events Compliance Management	Access and Compliance within the Manage stage	
Automated policy enforcement	Security policies can be managed in one place while compliant workflow automation helps admins easily apply compliance policies across projects.	Govern	Compliance pipelines
Audit reporting	GitLab tracks audit events across the entire SDLC and report them	Access and Compliance within the Manage stage	
Security Governance	Security Policy Automation, Compliant workflow automation, Security Dashboards and Vulnerability Reports, MR approvals, License compliance	Govern, Secure	
Security guardrails (Preventative - Pre CI/CD)	GitLab falls short of providing pre-approved dependencies as some other vendors do, bill of materials feature	Govern, Secure	Manage your Application Dependencies with GitLab

GitLab differentiators

Differentiator	Description
Single Application for Entire DevOps Lifecycle	a single application eliminates complex integrations, data chokepoints, and toolchain maintenance, resulting in greater productivity
End-to-End Insight and Visibility	GitLab's common data model enables end-to-end visibility and traceability throughout the DevOps lifecycle, correlating and aggregating data automatically
Deploy Your Software Anywhere	GitLab is infrastructure-agnostic (supporting GCP, AWS, Azure, OpenShift, VMWare, On Prem, Bare Metal, and more), offering a consistent workflow experience - irrespective of the environment
Leading SCM and CI in One Application	having the backbone of a DevOps toolchain in one application streamlines code review & collaboration (one interface, one user model, one data model)
Built-in Security and Compliance	move security testing earlier in the development lifecycle with out-of-the-box security features (code scanning, dependency scanning, secrets detection, etc.) and automated security testing and audit controls to facilitate policy compliance

Top Compliance differentiators

Differentiator	Value	Demo
MR approval based on Security Policy	Bring Development and Security Teams closer by allowing security teams to apply organizational security policies before hand and review/approve security exceptions before the code is merged	Merge-Request Approvals as Displayed in DevSecOps Overview
Compliance Management	GitLab makes compliance easier by providing a single source of truth for Dev, Sec and Ops through a single data-store. Everything is audited and for every change, there is a single thread that contains the full audit log of every decision and action - making audit compliance a breeze	Manage Compliance with GitLab
Compliant pipelines	Admin can choose a compliance framework and apply it to the project. It will override any changes developers make to the pipeline	Compliant pipelines

Message House

The message house for compliance provides a structure to describe and discuss the value and

differentiators for the Software compliance solution.

Top message: GitLab helps you take control of your software development with a single platform that helps you automate and standardize the development process and policies while providing end-to-end visibility/traceability so that development can run fast with less risk.

The GitLab DevOps platform approach helps you achieve Compliance with better visibility and control while simplifying audits and forensics. Governance is simplified with one place to manage policies, apply them, assess exceptions, and measure policy affect.

- A single platform improves allows the organization to see who changed what, where, and when across the entire SDLC.
- Compliance and Security policies can be automatically applied to all pipelines for consistency, and simplified audits
- Role-based access controls provide separation of duties and protect against malicious insider threats and accidental events
- A breadth of additional compliance controls provide superior governance over software development, delivery, and use.

Because developers see compliance concerns in the MR pipeline alongside security vulnerabilities, these can also be fixed while the developer is still iterating on the code, rather than waiting until pre-production when changes cost more time and money.

- Compliance concerns are shown for code changes and IaC
- Developers need not use another tool to identify compliance concerns
- Compliance is handled as part of the development process.

Competitive Comparison

See how we compare against other DevOps approaches

1. Role-based access control (RBAC) for separation of duties. Competitive products's roles are broader and when a person changes roles, his/her permissions must be changed manually. Why is this important? If someone has access to push to prod and is demoted or moves to another group, you'd want the permissions to change automatically to avoid insider threats.
1. Our workflows include compliance within MR approvals. No manual checks that impact velocity. (In essence, we shift left compliance also.)
1. External status checks is an important feature for regulated industries. Changes are approved and must be pushed to production within a given timeframe. Delays can cause the approval process to start over.
1. With GitLab we have projects and groups where projects inherit policies from the group. Competitors cannot structure policies as flexibly as GitLab, an important feature for enterprise users. Examples include group level runners.
1. Compliant pipelines allow GitLab users to select their compliance framework (e.g. PCI, HIPPA, etc) and those policies are used - and the developer cannot disable it (due to RBAC)

Key Value by tier

Free and Premium

Key Compliance features with Free/Premium:

- Security Approvals in Merge Requests (available in Premium)

In addition, some security scanning is available in the Free tier:

- Static Application Security Testing - check for potential security issues by evaluating static code.

- Secrets Detection - avoid exposing secrets and credentials for potential exploit.

Note: The Free tier does not include providing these results in vulnerability reports and dashboards. Raw findings are provided.

Ultimate

Key Compliance features with Ultimate:

- Compliant workflows and Compliant pipelines - projects are assigned a compliance framework and policies are enforced accordingly. This is a key capability for winning compliance opportunities!

- Compliance Dashboard - See if merge requests were approved, and by whom.

- Fuzz testing, called out by the 2021 US Executive Order to Improve Cybersecurity

- License Compliance - identify the presence of new software licenses included in your project and track project dependencies. Approve or deny the inclusion of a specific license.

In addition, more security scanners are available, along with Vulnerability management and security dashboard. See the DevSecOps solution for details.

Feature / Scenario	Free	Premium	Ultimate
Product Analytics Notes			
Compliant Pipelines	X		
Compliance Dashboard	X		
License Compliance			
X userlicensemanagementjobs			
Merge Request Approval Flow / Rules		X	X
counts.mergedmergerequestsusingapprovalrules			
Protected Environments		X	X
SAST (Static Application Security Testing)			X
X usersastjobs			
Secret Detection	X	X	X
usersecretDetectionjobs			
Container Scanning			X
usercontainerscanningjobs			
Dependency Scanning			X
userdependencyscanningjobs			
API Fuzzing			

are found late in the process | Strict change control means missed change windows when compliance issues are found late in the process | Strict change control means missed change windows when compliance issues are found late in the process |

Discovery Questions

Current state - where's the pain?

1. Simplicity

- How difficult is your compliance/audit process? (time, people, risk)
- Does it take away time from your developers?
- Are compliance issues found late in the software development/deployment process resulting in rework?

2. Visibility

- Do you have challenges between setting/automating policies and ensuring they were followed?
- Do you have visibility/traceability into audit events (e.g. access control, reporting, and change logs, etc.)?
- Can you see who made changes not only to the application code, but also to CI pipelines, configurations, templates, etc?
- Do you know how often policy exceptions are granted? How difficult is it to evaluate compliance with policies and drift/exceptions?

3. Control

- Are you able to ensure that policies are followed and not changed or ignored by developers or others?
- Is consistency a problem for security and compliance use? Are most projects secured a bit differently every time?
- If your software development does not comply with regulatory standards, what is the potential cost/risk?

4. For public sector and regulated industries only

- Do you often miss change management windows and have to reinitiate the change management process?

5. Related security questions

- What percentage of code are you currently scanning? Are there holes where an attacker could more easily enter and then traverse laterally? How much more would it cost you to scan all of your code?
- If you are using containers, orchestrators, and/or microservices/API's, how are you scanning them for vulnerabilities and monitoring them during production?
- How much of the security team's time is spent tracking vulnerabilities, triaging them, and following up to see that they were remediated?

Before state and negative consequences

| Before state | Negative consequences |

| ----- | ----- |

| 1. A complex tool chain makes it difficult to see who changed what, where, when across the entire SDLC | Audits are difficult and in the event of a breach, it could take days/weeks/months to identify root cause |

| 2. Audits are difficult and time consuming to show controls that cross multiple systems | A lack of traceability may require manual or other supplemental processes that are inefficient. Or you simply cannot show controls consistently across systems for a given project, opening yourself to risk. |

| 3. Non-compliance may be discovered right before production. | Rework and lost velocity. For Regulated industries, must restart change management process |

| 4. Policies are set but developers and other users can simply turn them off | This opens the door for security and compliance problems, including insider attacks. The lack of control lengthens audit efforts. |

| 5. Inconsistent pipelines across projects | Every project must be inspected uniquely for compliance. |

Future state - what's the desired state?

- Do you have targets in mind for how long it should take you to audit a project? What would that process look like?

- If you could ensure projects use a pipeline that is compliant with industry regulations, how much time/effort would that save you?

- If you could discover areas of noncompliance earlier, while the developer is still iterating on their code, how much rework would that eliminate or how would velocity be improved?

- If you could demonstrate automated controls for compliance, how would that simplify audits?

For public sector and regulated industries only

- If you could meet change management windows by having the compliance problems identified and worked out before the change window began, what would that save you in terms of rework? improved velocity?

After scenarios and PBOs

| After scenarios | Positive business outcomes |

| ----- | ----- |

| 1. End-to-end visibility of who changed what, where, when for application code, its dependencies, its IaC | Easier audits, less risk of insider threats, compromises more quickly discovered |

| 2. End-to-end consistent execution of policies across projects | Simplified audits and less risk |

| 3. Software development that is compliant with industry regulations | less risk of fines and other noncompliance consequences |

| 4. Discovery of compliance issues early, while developer is still iterating on the code | Less rework and therefore less developer time. For regulated industries, fewer missed change management windows. Improved velocity. |

Required capabilities

| Required capability | metrics |

| ----- | ----- |

| 1. End-to-end visibility of who changed what, where, when for application code, its dependencies, its IaC | time to identify root cause |

| 2. Policy enforcement for the entire SDLC from one console | time spent tracking exceptions and integrating policies between systems |

| 3. One role-based access control for the entire SDLC | time spent in work-arounds |

| 4. Inherited policies from instance to group to project | time spent setting up and reviewing compliance for individual projects |

| 5. Reporting of compliance concerns to the developer in the MR | time spent in rework late in the SDLC |

Overcoming Objections

- We can't afford to replace our current DevOps tools - We can meet you where you are and gradually work toward a smaller set of tools. Any tools eliminated is one less place to map and monitor compliance and security policies.

- We get by with our current processes - If you were breached, could you quickly identify potential sources of compromise? What is that worth to you?

- Others?

Proof Points - customers

Quotes and reviews

Customer Case Studies

Glympse

- Problem A complex developer tech stack with over 20 distinct tools that was hard to maintain and manage. Teams spent several hours a week keeping tools running rather than shipping innovation to their app.

- Solution: GitLab Ultimate (SCM, CI, DevSecOps)

- Results: ~20 tools consolidated into GitLab and remediated security issues faster than any other company in their Security Auditor's experience.

- Sales Segment: SMB

Chorus

- Problem: The founders of Chorus built the tool from the beginning using GitLab.

- Solution: GitLab Ultimate (SCM, CI, DevSecOps)

- Results: 6 week production cycles reduced to 1 week During a recent audit for SOC2 compliance, the auditors said that Chorus had the fastest auditing process they have seen and most of that is due to the capabilities of GitLab.

- Sales Segment: SMB

References to help you close

SFDC report of referenceable secure customers Note: Sales team members should have access to this report. If you do not have access, reach out to the customer reference team for

assistance.

Assets to share

Software compliance assets

- Blog: How a DevOps platform can help against supply chain attacks
- Blog: Compliance made easy with GitLab
- Blog: Three things you might not know about GitLab security
- Article: CI, the new frontier for security pros
- Commit 2021: Enhance your compliance with policy based CI/CD
- Webinar: From DevOps to DevSecOps: Automate your security tests with CI
- Webinar: Secure your software supply chain
- Demo: Compliant pipelines
- Demo: Manage your application dependencies with GitLab
- eBook: Guide to Securing your Software Supply Chain

Customer Facing Slides

- Continuous Software Compliance solution slides - feedback welcomed as comments in the deck.

Additional Documentation Links

- GitLab Security Compliance Controls
- GitLab Security Practices
- Security Planning

Enablement and Training

The following will link to enablement and training videos and content.

- Handling Security Audits
- Adding Security to your pipelines

Professional Service Offers

GitLab offers a variety of pre-packaged and custom services for our customers and partners. The following are service offers specific to this solution. For additional services, see the full service catalog.

- DevOps Fundamentals Training (all stages of the DevOps lifecycle)
- GitLab CI/CD Training
- Integration Services

Resources

Clickthrough & Live Demos

- All Marketing Click Through Demos
- All Marketing Live Demos

Roadmap

- Manage Direction

Technical Resources for Solution Architects - TBD

Buyer's Journey

Inventory of key assets in the buyer's Journey

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/usecase-gtm/delivery-automation.md

Looking for a customer-facing overview of GitLab's Automated Software Delivery Capabilities?

See Automated Software Delivery

The page below is intended to align GitLab sales and marketing efforts with a single source of truth for our go-to-market efforts around Automated Software Delivery.

Who to contact

	Product Marketing		Developer Advocate	
	-----		-----	
	Daniel Hom (@danielhom)		Aathira Nair (@anair5)	
	Itzik Gan Baruch (@iganbaruch)			

Background (2 min read)

Automated Software Delivery allows organizations to develop software faster by making teams more productive, unlocking operational efficiencies, accelerating app modernization, and unleashing the value of digital transformation.

- From a GitLab capabilities point of view - it includes DevOps essentials like SCM, CI, CD, GitOps, AutoDevOps and Observability

The opportunity is huge. As per the GitLab DevSecOps survey 2022 -

- 85% of organizations are using 2-10 DevOps tools
- 69% of organizations would like to consolidate their toolchains
- Devs are spending more time than ever before on maintaining or integrating toolchains. Nearly 40% said they spend between one-quarter and one-half of their time on these tasks (more than double the 2021 percentage), while 33% are spending at least half their time and as much as all of their time on toolchain integration and maintenance

Cloud transformation and application modernization is a key CIO initiative for 2022

- Automated Software Delivery becomes key to support the increase in cloud instances (microservices and ephemeral instances mean manually setting up, configuring and tearing down

these instances is not feasible anymore)

Getting Started {getting-started}

Who to meet / Ideal Customer Profile {who-to-meet}

The persona interested in application automation is generally different from the persona interested in infrastructure automation and continuous delivery. Application automation interest comes from development or engineering organizations while infrastructure automation and continuous delivery interest comes from the operations, systems, infrastructure, platform and cloud side of the organization.

| | Enterprise | Mid Market | SMB |
| - | ----- | ----- |
| User Persona | SCM/CI-> Developer, team lead, DevOps Engineer
 CD-> DevOps engineer, app operator, sys admin, release manager, platform engineer | SCM/CI-> Developer, team lead, DevOps Engineer
 CD-> DevOps engineer, app operator, sys admin, release manager, platform engineer | varies |
| Buyer Persona | CIO / CTO at C level
 SCM/CI -> Mgr / Director - App Dev
 CD -> Mgr / Director - Infra/Ops | CIO / CTO at C level
 SCM/CI -> Mgr / Director - App Dev
 CD -> Mgr / Director - Infra/Ops | varies |
| Tech Stack | Includes GitHub, Perforce, Jenkins, BitBucket, Subversion, ArgoCD, Harness | Includes GitHub, Perforce, Jenkins, BitBucket, Bamboo, Subversion, ArgoCD, Harness OR the absence of a tech stack OR freemium version of GitLab / GitHub | Absence of tech stack OR freemium version of GitLab / GitHub |
| Infra / Cloud Provider / Deploy Targets | AWS or GCP or internal data center or Kubernetes | AWS or GCP (or none) or Kubernetes | AWS or GCP (or none) or Kubernetes |
| Additional Criteria | - Automation a key initiative
 - merger & acquisition
 - consolidation of CI-CD solutions
 - increased hiring and projects
 - hiring new CIO, App Dev, Infra teams | - Automation a key initiative
 - increased hiring and projects
 - hiring new CIO, App Dev, Infra teams | - increased hiring and projects
 - adoption of new cloud/technologies |
| Business Driver | - Digital Transformation
 - Cloud Transformation
 - Application Modernization
 - Automated Software Delivery | - Cloud Transformation
 - Application Modernization
 - Automated Software Delivery | - Cloud Transformation
 - Application Modernization
 - Automated Software Delivery |

Keywords to listen for {keywords}

- Manual efforts
 - including Manual builds, Manual code reviews, Manual peer reviews, Manual testing, manual deployments, manual infrastructure configurations, amongst others.
- Employee dissatisfaction
 - listen to Repetitive tasks, too much time in managing pipelines, setting up and configuring pipelines, setting up and configuring infrastructure, managing releases
- Increase speed / velocity
- Cloud native applications
- Kubernetes and containers
- Multi platform / multi language support
- Cloud transformation

- Application Modernization

Value discovery {value-discovery}

Discovery Questions · Current State: Where's the pain?

1. How much of your team's job is manual? This should help scope interest. Many teams struggle with little or no automation - resulting in inefficiencies, errors and employee dissatisfaction. Key areas of manual effort include - unit testing, performance testing, code review, peer reviews, license checks, manual transfer of application images, manual deploy, manual post deploy testing, amongst others.

1. How much time is your team spending in configuring, fixing, or maintaining your pipelines compared to doing "real work"? Managing complex pipelines is a real problem. We find that our customers with complicated toolchains have more difficulty managing complex pipelines and supporting integrations. This usually makes implementing and maintaining CI/CD more expensive than originally planned.

1. Are you able to manage both current and latest technologies with your existing toolset? Moving to cloud or cloud native technologies like Kubernetes requires a different way of operation - their existing toolset may not be built ground up for such transformations and could require hacking a number of fragile integrations together to support.

1. Are you able to measure key success metrics for your automation? Metrics such as Change Lead Time, Deployment Frequency, Mean Time to Resolution, and Change Failure Rate. This will reveal the lack of integration and traceability across various tools being put together to achieve automation.

1. Are you able to deploy with confidence? Are you able to have full visibility into the pipeline from idea to production? Are you able to setup your infra, deploy in phases, test, roll back on error - all from a single application?

Discovery Questions · Future State: Where Would You Like to Be?

1. What does automation success look like? What parts of your SDLC would you have fully automated? This can help uncover areas that they immediately see a pain and can help you identify areas of growth for the customer

1. What would your team do with the time reclaimed from maintaining pipelines? This provides an opportunity to discuss what true success would look and how freeing resources improves productive value added by the team

1. What technologies do you see being supported by your CI-CD solution? This can help uncover future technology adoption and drivers for change. They may also have a need to have various technologies being supported by the same solution

1. What success metrics would you want to showcase? What are the short term metrics and long term metrics? This can help uncover areas of success we can immediately showcase

1. How important are safe, progressive deployments to your organization? This uncovers the importance of downtime and how flexible deployment options are key to their automation success.

Common Pains Points {common-pains}

| Challenges "before scenarios" | So What? "negative consequences" |

| ----- | ----- |

| Manual processes, lack of standardization | Highly inefficient and error prone processes |

| Employee dissatisfaction due to manual and repetitive processes | Employee churn, inability

to hire & retain talent |

| Fragile, complex pipelines are tough to maintain | Requires expert, expensive team members who become a single point of failure |

| Adoption of new technologies not fast enough | Requires new tools, expertise for managing new technologies |

| Infrastructure configuration not fast enough or repeatable | Lack of standardization and automation leads to higher downtime and errors |

| Lack of scalable review and authorization processes | Highly skilled workers doing menial tasks - leading to dissatisfaction |

| Lack of safe deployment options and traceability | Longer time to repair, downtime and customer perception issues |

Common benefits {common-benefits}

| Desired Future State ("After Scenarios") | So What? ("Positive Business Outcomes") |

| ----- | ----- |

| Faster time to value with automation and standardization of testing, review, packaging, release and deployment | Automation to focus on "real work", less risk, improved employee satisfaction |

| Faster mean time to repair with better traceability and deployment options | Reduce customer perception issues, downtime, improve retention |

| Faster adoption of cloud native technologies | Cloud native first development along with current deployment targets for more flexibility using a single tool |

| Better employee satisfaction | Improved automation, removal of manual / repetitive tasks and reduce integration work improves employee satisfaction |

Required capabilities {required-capabilities}

| Required capability | Customer Metrics |

| ----- | ----- |

| Test automation, pipeline configuration management | Automation to focus on "real work", less risk, improved employee satisfaction |

| Change control and collaboration | Reduce customer perception issues, downtime, improve retention |

| Safe deployment and roll back | Cloud native first development along with current deployment targets for more flexibility using a single tool |

| Code quality, peer reviews | Improved automation, removal of manual / repetitive tasks and reduce integration work improves employee satisfaction |

Customer Adoption and Value

These value statements are used to measure of GitLab Adoption by the customer and provide guidance to our field teams (CSMs in particular) for conversations on expanding usage.

CI Adoption Score

| Area | Measure | Value |

| -- | -- | -- |

| CI Utilization | Depth of CI Adoption as a percentage of licensed seats | - Increase build speeds. Development teams work more efficiently by reducing otherwise manual work.
 - Catch

potential errors sooner rather than later before they intensify.
 - Maximize development time and improves productivity. Less manual work.
 - Faster feedback and less risk that changes cause builds to break.
 - Gives teams more flexibility, making it easier to adopt.
 - Reduces business risk and protects intellectual property. Instills confidence in end-users. |

| Container Registry | Container registry enabled | - Centralize the publication and consumption of container images
 - Standardize on authentication methods and authorization models throughout your software lifecycle
 - Eliminate the need to store & rotate credentials for third-party systems (i.e. leverage CI Job Tokens with GitLab CI to authenticate)
 - Ensure standardized use of cloud object storage to limit attack surface (eg. avoid leaks through open S3 buckets)
 - Improve visibility and control over cloud storage utilization on a per-project/per-group basis (i.e. through storage quotas, expiration policies, and usage reporting) |

| Templates | The number of instance-wide template repositories that exist in an instance | - Proliferate best-practices approaches through easily reusable pipeline templates
 - Promote innersourcing and collaboration through the use of templates
 - Speed time to green pipeline by providing new projects a well defined starting point
 - Improve compliance with required reporting, or other important pipeline tasks (test coverage, linting, scanning, etc.) |

| Deployments Utilization | Depth of Deployment adoption as a percentage of licensed seats | - Faster time to value with automation and standardization of testing, review, packaging, release and deployment
 - Faster mean time to repair with better traceability and deployment options
 - Cloud native first development along with current deployment targets for more flexibility using a single tool |

| Deployment Count per User L28D | Deployment Count per User (last 28 days)
 - Number of deployments within last 28D divided by number of licensed users | - Faster time to value with automation and standardization of testing, review, packaging, release and deployment
 - Faster mean time to repair with better traceability and deployment options
 - Cloud native first development along with current deployment targets for more flexibility using a single tool |

| Value Stream analytics | How many users went to the VSA page in the last 28 days | - VSA helps businesses identify and target waste and inefficiencies, and take action to optimize those workstreams to deliver the highest possible velocity of value.
 - GitLab's one platform approach ensures a single platform, workspace and data model which creates a shared view of actionable insights, of your end to end software delivery lifecycle |

Positioning value {positioning-value}

Elevator pitch

GitLab enables you to deliver better software, faster. Unleash team productivity and efficiency, and accelerate the velocity of software development by automating the build, test, and deployment of your applications. Accelerate your digital transformation and app modernization goals with automated provisioning and maintenance of cloud-native infrastructure. Finally, GitLab allows you to easily start where you are, and reduce tooling and complexity as your adoption matures with our flexible platform that has built-in, end-to-end security and out-of-the-box integrations

Value Proposition (How GitLab does it?)

- GitLab helps you deliver better software, faster

> "Instead of spending a full day setting up for production and doing manual tests, those simple tasks are now automated. This allowed release time to go from 24 hours to just 10 minutes." - Airbus

- GitLab is one of the easiest platforms to get started with your DevOps journey

> "It just worked. That was the biggest success because we didn't have any problems with installing GitLab, with updating it and it was really simple to start for us," [MGA]

- GitLab allows you to start where you are in your journey and reduce other tools as your adoption matures

> "As per the Forrester Total Economic Impact study, customers were able to eliminate at least 3 tools per year" TEI

- GitLab supports your cloud-native application transformations (as well as your traditional applications!)

> "GitLab integrates natively with Kubernetes, which gives the development team peace of mind because they can trust that the tool will work automatically without constant maintenance." - Hotjar

- GitLab allows you to automate infrastructure and deploy to different cloud providers (and your existing infrastructure as well)

> "Our DevOps teams can provision production-ready solutions within minutes, from Kubernetes to multi-cloud and multi-tier complex hybrid cloud infrastructures" - SURF

- GitLab is a single application for full Automated Software Delivery including change management, CI/CD and infrastructure automation allowing you to achieve better stability and ship high quality applications faster

> "If something goes wrong, GitLab offers a simple fix. For example, if we deployed the wrong ratio of our services to the server, we can see the history, read the commit message, and identify the author to roll back commits" - Parimatch

- GitLab helps improve developer experience

> "Developer happiness matters because happy developers simply do better work. You need to keep your developer experience at a high level because that's the only way you really can deliver value fast. GitLab has played a significant part in helping us increase our developer experience." - Moneyfarm

Differentiators (How GitLab does Automated Software Delivery better?)

Apart from the single application, DevOps Platform narrative, there are a number of key areas which GitLab does well and better than competitors.

Differentiator	Description
Multi-platform	Execute builds on Unix, Windows, OSX, and any other platform that supports Go
Multi-language	Build scripts work with Java, PHP, Ruby, C, and any other language
Multi-deployment targets	Deploy to embedded systems, on-premise servers, Raspberry Pi devices, mainframes, virtual machines, Kubernetes clusters, FaaS offerings, multiple clouds like AWS, GCP, Azure, IBM and Oracle
Faster builds	GitLab splits builds over multiple machines for parallel execution
Autoscaling	Automatically spin up and down VM's or Kubernetes pods to make sure your builds get processed immediately while minimizing costs
Versioned tests	a .gitlab-ci.yml file that contains your tests, allowing developers to contribute changes and ensuring every branch gets the tests it needs
Flexible Pipelines	Define multiple jobs per stage and even trigger other pipelines
Ease test environment creation	With the review app - use custom Docker images, spin up services as part of testing, build new Docker images, run on Kubernetes - use this to run UX tests, DAST, usability tests etc
Built in Container Registry	built-in container registry to store, share, and use container images
Distributed version control	Reduced time to clone and fetch large repos for geographically distributed teams
Product development management	Version control not just source code but also IP, graphic assets, animations and binaries to mention a few
Unified deployment and observability	Visualise what goes into production (Review apps and release planning), what to deploy to production (feature flags) , who to deploy it to (Progressive Delivery and deployment strategies like Canary), monitor performance of deployment (via browser performance testing, performance monitoring/tracing) and rollback based on performance via post deployment monitoring, all from a single application
Secure Kubernetes Cluster access	Avoid exposing your cluster by establishing a secure connection with your Kubernetes cluster for your CI/CD jobs
Push and pull based deployments	Majority of customers still prefer the control that push based deployment gives, we support both

Competitors

From a competitive comparison for Automated Software Delivery, the following are the key competitors and what they are lacking at a high level. Review the detailed comparison in the devops-tools comparison pages

- GitHub - Strong on SCM, GitHub Actions for CI/CD is still not ready for primetime. GitLab can integrate and migrate customers using GitHub for SCM easily with GitLab.
- Jenkins - Complex to manage, many fragile integrations, not suitable for cloud native transformation
- JFrog - Strong on CI, but requires integrations or has limited capabilities for rest of the areas.
- Circle CI
- Weaveworks Flux CD - Main solution only for the GitOps operator, requires multiple integrations to achieve end to end GitOps flow.

Handling Objections

| Objection | Response |

| ----- | ----- |

| "I don't need another tool to replace my existing tool" | - We can meet you where you are - you don't need to rip and replace all your existing tools
 - You can start by supplementing GitLab CI or CD or SCM with your existing toolset of Jira, GitHub, Jenkins etc
 - Once you find value in the specific area (e.g., SCM, CI, CD, Security, Compliance) - you can expand your use of GitLab to achieve better ROI |

| "I don't want to be locked into one company for my entire DevOps lifecycle" | - Toolchain sprawl is a reality - many customers have stopped benefitting from DevOps due to this
 - Application & resource integration & management actually hinders how fast the organization can move.
 - Customers partner with GitLab to accelerate time to market for new products/services, improve efficiencies & reduce security & compliance risks. Walk me through how your priorities align with these outcomes. |

| "Each development team decides what they need" | - Allowing each team to decide creates silos of innovation & development. It also significantly limits the impact of your digital transformation.
 - Each team then also needs to setup, integrate and maintain these integrations.
 - Describe how this works for you. How many tools have you spent money on